



Nama/NIM : Claudya Destine Julia Handoko/2341760008
Mata Kuliah : Pemrograman Web Lanjut (PWL)
Program Studi : D4 – Teknik Informatika / D4 – Sistem Informasi Bisnis
Semester : 4 (empat) / 6 (enam)
Pertemuan ke- : 10 (tujuh)

JOBSHEET 10

RESTFUL API

Sebelumnya kita sudah membahas mengenai *authentication*, *authorization*, dan *middleware* pada Laravel. Dimana kita telah membuat fungsi login, register, logout, serta pemilihan role dan penerapan session pada halaman web. Pada pertemuan kali ini, kita akan mempelajari penerapan RESTFUL API di dalam project Laravel.

Sebelum kita masuk materi, kita buat dulu project baru yang akan kita gunakan untuk membangun aplikasi sederhana dengan topik *Point of Sales (PoS)*, sesuai dengan **Studi Kasus PWL.pdf**. Jadi kita bikin project Laravel 10 dengan nama **PWL_POS**.

Project PWL_POS akan kita gunakan sampai pertemuan 12 nanti, sebagai project yang akan kita pelajari

A. RESTFUL API

Representational State Transfer (REST) adalah gaya arsitektur perangkat lunak yang mendefinisikan seperangkat prinsip untuk merancang jaringan aplikasi terdistribusi. RESTful API adalah aplikasi pemrograman antarmuka yang mengikuti prinsip-prinsip REST untuk mentransfer data antara klien dan server.

RESTful API adalah salah satu arsitektur dalam API (*Application Program Interface*) yang menggunakan request HTTP untuk mengakses data. Data diakses dengan menggunakan HTTP method GET, PUT, POST dan DELETE yang merujuk pada operasi pembacaan, pembaruan, pembuatan dan penghapusan pada resource. Selain HTTP method, dalam RESTful atau REST digunakan juga HTTP response untuk mendefinisikan respon data yang dikembalikan. Format respon yang umum digunakan berupa JSON (Javascript Object Notation).



B. JSON Web Token (JWT)

JWT adalah singkatan dari JSON Web Token. Ini adalah standar terbuka (RFC 7519) yang mendefinisikan format token yang kompak dan mandiri untuk mentransfer klaim antara dua pihak. JWT sering digunakan dalam otentikasi dan pertukaran informasi yang aman di lingkungan yang tidak terpercaya, seperti internet.

JWT terdiri dari tiga bagian yang dipisahkan oleh titik ("."): header, payload, dan signature. Setiap bagian ini terdiri dari data JSON yang dienkripsi menggunakan algoritma tertentu dan kemudian disatukan untuk membentuk token yang lengkap. Header berisi jenis token dan tipe algoritma yang digunakan untuk enkripsi. Payload berisi klaim atau informasi yang ingin disampaikan. Signature digunakan untuk memverifikasi bahwa token belum berubah dan datanya berasal dari sumber yang dipercaya.

JWT sering digunakan dalam sistem otentikasi dan otorisasi modern, seperti aplikasi web dan layanan web API, karena fleksibilitasnya dalam menyampaikan informasi terenkripsi secara ringkas.

Kita dapat menggunakan JWT untuk:

- **Authentication**
Ketika pengguna melakukan authentication dan mendapatkan token, maka setiap permintaan berikutnya akan menyertakan token tersebut, dan memungkinkan pengguna untuk mengakses route, service, dan resources yang diizinkan.
- **Pertukaran informasi**
JSON Web Token adalah cara yang baik untuk mengirimkan informasi antar pihak dengan aman. Dengan token yang sudah ditandatangani dengan algoritma RSA, maka kita bisa tahu siapa yang melakukan request tersebut.

Berikut adalah cara kerja JWT :

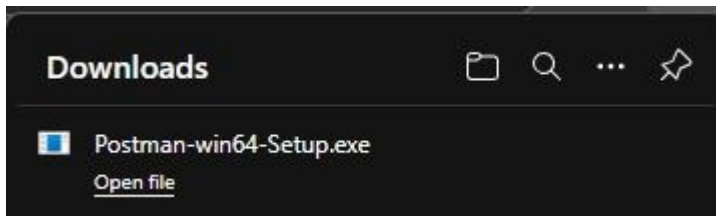
JWT (JSON Web Token) adalah cara untuk mentransfer informasi antara dua pihak secara aman sebagai objek JSON. Ini terdiri dari tiga bagian: header, payload, dan signature. Setelah pengguna berhasil autentikasi, server menghasilkan token JWT yang disematkan dalam permintaan HTTP. Server kemudian memvalidasi token untuk memberikan akses ke sumber daya yang diminta. Ini memberikan autentikasi yang aman dan stateless tanpa memerlukan penyimpanan status sesi di server.

Praktikum 1 – Membuat RESTful API Register



1. Sebelum memulai membuat REST API, terlebih dahulu download aplikasi Postman di <https://www.postman.com/downloads>.

Aplikasi ini akan digunakan untuk mengerjakan semua tahap praktikum pada Jobsheet ini.



2. Lakukan instalasi JWT dengan mengetikkan perintah berikut: `composer require tymon/jwt-auth:2.1.1` Pastikan Anda terkoneksi dengan internet.

```
PS C:\laragon\www\PHL2025\Week10\Jobsheet10> composer require tymon/jwt-auth:2.1.1
./composer.json has been updated
Running composer update tymon/jwt-auth
Loading composer repositories with package information
Updating dependencies
Lock file operations: 4 installs, 0 updates, 0 removals
- Locking lcobucci/clock (2.3.0)
- Locking lcobucci/jwt (4.0.4)
- Locking stella-maris/clock (0.1.7)
- Locking tymon/jwt-auth (2.1.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 6 installs, 0 updates, 0 removals
- Downloading stella-maris/clock (0.1.7)
- Downloading lcobucci/clock (2.3.0)
- Downloading phpoffice/phpexcel (1.8.1)
- Downloading maatwebsite/excel (v1.1.5)
- Downloading lcobucci/jwt (4.0.4)
- Downloading tymon/jwt-auth (2.1.1)
- Installing stella-maris/clock (0.1.7): Extracting archive
- Installing lcobucci/clock (2.3.0): Extracting archive
- Installing phpoffice/phpexcel (1.8.1): Extracting archive
- Installing maatwebsite/excel (v1.1.5): Extracting archive
- Installing lcobucci/jwt (4.0.4): Extracting archive
- Installing tymon/jwt-auth (2.1.1): Extracting archive
```



```
INFO Discovering packages.

barryvdh/laravel-dompdf ..... DONE
jeroennoten/laravel-adminlte ..... DONE
laravel/sail ..... DONE
laravel/sanctum ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE
spatie/laravel-ignition ..... DONE
tymon/jwt-auth ..... DONE
yajra/laravel-datatables-buttons ..... DONE
yajra/laravel-datatables-editor ..... DONE
yajra/laravel-datatables-fractal ..... DONE
yajra/laravel-datatables-html ..... DONE
yajra/laravel-datatables-oracle ..... DONE

93 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

INFO No publishable resources for tag [laravel-assets].

Found 1 security vulnerability advisory affecting 1 package.
Run "composer audit" for a full list of advisories.
```

3. Setelah berhasil menginstall JWT, lanjutkan dengan publish konfigurasi file dengan perintah berikut:

```
php artisan vendor:publish --
provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"
```

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan vendor:publish --provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"

INFO Publishing assets.

Copying file [C:\laragon\www\PWL2025\Week10\Jobsheet10\vendor\tymon\jwt-auth\config\config.php] to [C:\laragon\www\PWL2025\Week10\Jobsheet10\config\jwt.php] DONE
```

4. Jika perintah di atas berhasil, maka kita akan mendapatkan 1 file baru yaitu config/jwt.php. Pada file ini dapat dilakukan konfigurasi jika memang diperlukan.

5. Setelah itu jalankan perintah berikut untuk membuat secret key JWT. `php artisan jwt:secret`

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan jwt:secret
jwt-auth secret [5nXwIhNTEwy6puOtFAnCK0YZXPERLi13cIG8GdM1rTLmk6azG9F0yPsEy6JT6ah] set successfully.
```

Jika berhasil, maka pada file .env akan ditambahkan sebuah baris berisi nilai key JWT_SECRET.

```
61 JWT_SECRET=5nXwIhNTEwy6puOtFAnCK0YZXPERLi13cIG8GdM1rTLmk6azG9F0yPsEy6JT6ah
62
```



6. Selanjutnya lakukan konfigurasi guard API. Buka config/auth.php. Ubah bagian 'guards' menjadi seperti berikut.

```
'guards' => [  
    'web' => [  
        'driver' => 'session',  
        'provider' => 'users',  
    ],  
    'api' => [  
        'driver' => 'jwt',  
        'provider' => 'users',  
    ],  
],
```

Hasil:

```
38  ✓ 'guards' => [  
39  ✓     'web' => [  
40      'driver' => 'session',  
41      'provider' => 'users',  
42      ],  
43  ✓     'api' => [  
44      'driver' => 'jwt',  
45      'provider' => 'users',  
46      ],  
47  ✓ ],
```

7. Kita akan menambahkan kode di model UserModel, ubah kode seperti berikut:



```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Tymon\JWTAuth\Contracts\JWTSubject;
use Illuminate\Foundation\Auth\User as Authenticatable;

class UserModel extends Authenticatable implements JWTSubject
{

    public function getJWTIdentifier(){
        return $this->getKey();
    }

    public function getJWTCustomClaims(){
        return [];
    }

    protected $table = 'm_user';
    protected $primaryKey = 'user_id';
```

Hasil :

```
app > Models > UserModel.php > PHP Intelephense > UserModel

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Factories\HasFactory;
7  use Illuminate\Database\Eloquent\Relations\BelongsTo;
8  use Illuminate\Foundation\Auth\User as Authenticatable;
9  use Tymon\JWTAuth\Contracts\JWTSubject; // <--- Tambahkan ini
10
11  21 references | 0 implementations
12  class UserModel extends Authenticatable implements JWTSubject // <--- Implement JWTSubject
    {
        public function getJWTIdentifier(): mixed
        {
            return $this->getKey();
        }

        /**
         * Return a key value array, containing any custom claims to be added to the JWT.
         */
        0 references | 0 overrides
        public function getJWTCustomClaims(): array
        {
            return [];
        }
    }
```



8. Berikutnya kita akan membuat controller untuk register dengan menjalankan perintah berikut.

```
php artisan make:controller Api/RegisterController
```

Jika berhasil maka akan ada tambahan controller pada folder Api dengan nama RegisterController.

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/RegisterController

INFO Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\RegisterController.php] created successfully.
```

9. Buka file tersebut, dan ubah kode menjadi seperti berikut.

```
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Models\UserModel;
6  use App\Http\Controllers\Controller;
7  use Illuminate\Http\Request;
8  use Illuminate\Support\Facades\Validator;
9
10 class RegisterController extends Controller
11 {
12     public function __invoke(Request $request)
13     {
```




```
14 //set validation
15 $validator = Validator::make($request->all(), [
16     'username' => 'required',
17     'nama' => 'required',
18     'password' => 'required|min:5|confirmed',
19     'level_id' => 'required'
20 ]);
21
22 //if validations fails
23 if($validator->fails()){
24     return response()->json($validator->errors(), 422);
25 }
26
27 //create user
28 $user = UserModel::create([
29     'username' => $request->username,
30     'nama' => $request->nama,
31     'password' => bcrypt($request->password),
32     'level_id' => $request->level_id,
33 ]);
34
35 //return response JSON user is created
36 if($user){
37     return response()->json([
38         'success' => true,
39         'user' => $user,
40     ], 201);
41 }
42
43 //return JSON process insert failed
44 return response()->json([
45     'success' => false,
46 ], 409);
47 }
48 }
```

Hasil :



```
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use Illuminate\Http\Request;
7  use App\Models\UserModel;
8  use Illuminate\Support\Facades\Hash;
9  use Illuminate\Support\Facades\Validator;
10
11 class RegisterController extends Controller
12 {
13     public function register(Request $request)
14     {
15         // Validasi input
16         $validator = Validator::make($request->all(), [
17             'username' => 'required|string|unique:m_user,username',
18             'nama' => 'required|string|max:255',
19             'password' => 'required|string|min:5',
20         ]);
21
22         if ($validator->fails()) {
23             return response()->json($validator->errors(), 422);
24         }
25
26         // Simpan user baru
27         $user = UserModel::create([
28             'username' => $request->username,
29             'nama' => $request->nama,
30             'password' => Hash::make($request->password),
31             'level_id' => 2, // Default level_id kalau mau (sesuaikan dengan kebutuhanmu)
32         ]);
33
34         return response()->json([
35             'status' => true,
36             'message' => 'Registrasi berhasil!',
37             'data' => $user
38         ], 201);
39     }
40 }
```

10. Selanjutnya buka routes/api.php, ubah semua kode menjadi seperti berikut.



```
<?php

use App\Http\Controllers\Api\RegisterController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

/*
|-----
| API Routes
|-----
|
| Here is where you can register API routes for your application. These
| routes are loaded by the RouteServiceProvider and all of them will
| be assigned to the "api" middleware group. Make something great!
|
*/

Route::post('/register', App\Http\Controllers\Api\RegisterController::class)->name('register');
```

Hasil :

```
routes > api.php
1  <?php
2
3  use Illuminate\Http\Controllers\Api\RegisterController;
4  use Illuminate\Http\Request;
5  use Illuminate\Support\Facades\Route;
6
7
8  /*
9  |-----
10 | API Routes
11 |-----
12 |
13 | Here is where you can register API routes for your application. These
14 | routes are loaded by the RouteServiceProvider and all of them will
15 | be assigned to the "api" middleware group. Make something great!
16 |
17 */
18
19 Route::post('/register', App\Http\Controllers\Api\RegisterController::class)
20 ->name('register');
21
22
```

11. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman. Buka aplikasi Postman, isi URL `localhost/PWL_POS/public/api/register` serta method POST. Klik Send.



POST localhost/PWL_POS/public/api/register Send

Params Auth Headers (7) Body Pre-req. Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body 422 Unprocessable Content 272 ms 534 B Save Response

Pretty Raw Preview Visualize JSON

```
1 2
2  "username": [
3    "The username field is required."
4  ],
5  "nama": [
6    "The nama field is required."
7  ],
8  "password": [
9    "The password field is required."
10 ]
11
```

Jika berhasil akan muncul error validasi seperti gambar di atas.

Lakukan percobaan yang sama dan berikan screenshot hasil percobaan Anda.



The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://127.0.0.1:8000/api/register
- Status:** 422 Unprocessable Content
- Response Body (JSON):**

```
1 {
2   "username": [
3     "The username field is required."
4   ],
5   "nama": [
6     "The nama field is required."
7   ],
8   "password": [
9     "The password field is required."
10  ]
11 }
```

12. Sekarang kita coba masukkan data. Klik tab Body dan pilih form-data. Isikan key sesuai dengan kolom data, serta isikan data registrasi menggunakan nilai yang Anda inginkan.



POST localhost/PWL_POS/public/api/register Send

Params Auth Headers (8) **Body** Pre-req. Tests Settings Cookies

form-data

	Key	Value	...	Bulk Edit
☒	username	penggunasatu		
☒	nama	Pengguna 1		
☒	password	12345		
☒	password_confirmation	12345		
☒	level_id	2		

body Cookies Headers (11) Test Results 201 Created 624 ms 645 B Save Response

Pretty Raw Preview Visualize JSON

```
1
2  "success": true,
3  "user": {
4    "username": "penggunasatu",
5    "nama": "Pengguna 1",
6    "password": "$2y$12$Eb2SrV1jsykINyTYGtrHi0DVAKcK5p6EgnZnmbChkPicIu7S0QJJJu",
7    "level_id": "2",
8    "updated_at": "2024-04-22T15:56:04.000000Z",
9    "created_at": "2024-04-22T15:56:04.000000Z",
10   "user_id": 17
```

Setelah klik tombol Send, jika berhasil maka akan keluar pesan sukses seperti gambar di atas.

Lakukan percobaan yang sama dan berikan screenshoot hasil percobaan Anda.

Home Workspaces API Network Search Postman Ctrl K Invite Upgrade

Overview Getting started POST http://127.0.0.1:8000/a

http://127.0.0.1:8000/api/register Save Share

POST http://127.0.0.1:8000/api/register Send

Params Authorization Headers (9) **Body** Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

	Key	Value	Description	...	Bulk Edit
☒	username	AdminClaudia			
☒	nama	Cloud			
☒	password	12345678			
☒	password_confirmation	12345678			
☒	level_id	1			



Body 201 Created • 2.28 s • 493 B •

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "user": {
4     "username": "AdminClaudya",
5     "nama": "Cloud",
6     "level_id": "1",
7     "updated_at": "2025-04-30T13:23:14.000000Z",
8     "created_at": "2025-04-30T13:23:14.000000Z",
9     "user_id": 33
10  }
11 }
```

17	AdminClaudya	Cloud	Administrator	Detail	Edit	Hapus
----	--------------	-------	---------------	------------------------	----------------------	-----------------------

13. Lakukan commit perubahan file pada Github.

Praktikum 2 – Membuat RESTful API Login

1. Kita buat file controller dengan nama LoginController. `php artisan make:controller Api/LoginController`

Jika berhasil maka akan ada tambahan controller pada folder Api dengan nama LoginController.

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/LoginController

INFO Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api>LoginController.php] created successfully.
```

2. Buka file tersebut, dan ubah kode menjadi seperti berikut.

```
1 <?php
2
3 namespace App\Http\Controllers\Api;
4
5 use App\Http\Controllers\Controller;
6 use Illuminate\Http\Request;
7 use Illuminate\Support\Facades\Validator;
8
```



```
9 class LoginController extends Controller
10 {
11     public function __invoke(Request $request)
12     {
13         //set validation
14         $validator = Validator::make($request->all(), [
15             'username' => 'required',
16             'password' => 'required'
17         ]);
18
19         //if validation fails
20         if ($validator->fails()) {
21             return response()->json($validator->errors(), 422);
22         }
23
24         //get credentials from request
25         $credentials = $request->only('username', 'password');
26
27         //if auth failed
28         if (!$token = auth()->guard('api')->attempt($credentials)) {
29             return response()->json([
30                 'success' => false,
31                 'message' => 'Username atau Password Anda salah'
32             ], 401);
33         }
34
35         //if auth success
36         return response()->json([
37             'success' => true,
38             'user' => auth()->guard('api')->user(),
39             'token' => $token
40         ], 200);
41     }
42 }
```

Hasil :

```
app > Http > Controllers > Api > LoginController.php > PHP > LoginController > login()
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use Illuminate\Http\Request;
7  use Illuminate\Support\Facades\Validator;
8
9  0 references | 0 implementations
9  class LoginController extends Controller
10 {
11
12  0 references | 0 overrides
```




```
app > Http > Controllers > Api > LoginController.php > PHP Intelephense > LoginController > _invoke
  references | 0 implementations
9  class LoginController extends Controller
10 {
11     0 references | 0 overrides
12     public function __invoke(Request $request): JsonResponse|mixed
13     {
14         // Set validation
15         $validator = Validator::make($request->all(), [
16             'username' => 'required',
17             'password' => 'required',
18         ]);
19
20         // If validation fails
21         if ($validator->fails()) {
22             return response()->json($validator->errors(), 422);
23         }
24
25         // Get credentials from request
26         $credentials = $request->only('username', 'password');
27
28         // If auth failed
29         if (!$token = auth()->guard('api')->attempt(credentials: $credentials)) {
30             return response()->json([
31                 'success' => false,
32                 'message' => 'Username atau Password Anda salah'
33             ], 401);
34         }
35
36         // If auth success
37         return response()->json([
38             'success' => true,
39             'user' => auth()->guard('api')->user(),
40             'token' => $token
41         ], 200);
42     }
43 }
```

3. Berikutnya tambahkan route baru pada file api.php yaitu /login dan /user.

```
use App\Http\Controllers\Api\LoginController;

Route::post('/register', App\Http\Controllers\Api\RegisterController::class)->name('register');
Route::post('/login', App\Http\Controllers\Api\LoginController::class)->name('login');
Route::middleware('auth:api')->get('/user', function (Request $request) {
    return $request->user();
});
```

Hasil :



```
routes > api.php > ...
1  <?php
2
3  use App\Http\Controllers\Api\RegisterController;
4  use Illuminate\Http\Request;
5  use Illuminate\Support\Facades\Route;
6  use App\Http\Controllers\Api>LoginController;
7
8
9  /*
10 |-----
11 | API Routes
12 |-----
13 |
14 | Here is where you can register API routes for your application. These
15 | routes are loaded by the RouteServiceProvider and all of them will
16 | be assigned to the "api" middleware group. Make something great!
17 |
18 |*/
19
20 Route::post('/register', App\Http\Controllers\Api\RegisterController::class)->name('register');
21 Route::post('/login', App\Http\Controllers\Api>LoginController::class)->name('login');
22 Route::middleware('auth:api')->get('/user', function (Request $request): mixed {
23 |     return $request->user();
24 | });
```

4. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman. Buka aplikasi Postman, isi URL localhost/PWL_POS/public/api/login serta method POST. Klik Send.

POST localhost/PWL_POS/public/api/login Send

Params Auth Headers (7) Body Pre-req. Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

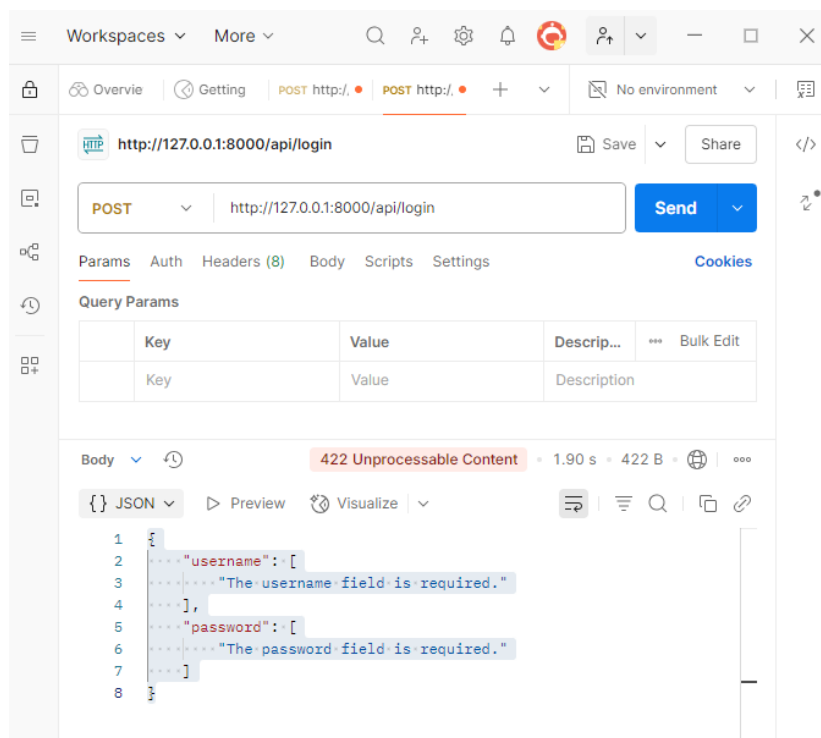
Body Cookies Headers (11) Test Results 422 Unprocessable Content 563 ms 495 B Save Response

Pretty Raw Preview Visualize JSON

```
1  {
2    "username": [
3      "The username field is required."
4    ],
5    "password": [
6      "The password field is required."
7    ]
8  }
```

Jika berhasil akan muncul error validasi seperti gambar di atas.

Lakukan percobaan yang sama dan berikan screenshoot hasil percobaan Anda.



5. Selanjutnya, isikan username dan password sesuai dengan data user yang ada pada database. Klik tab Body dan pilih form-data. Isikan key sesuai dengan kolom data, serta isikan data user. Klik tombol Send, jika berhasil maka akan keluar tampilan seperti berikut. Copy nilai token yang diperoleh pada saat login karena akan diperlukan pada saat logout.



POST localhost/PWL_POS/public/api/login Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

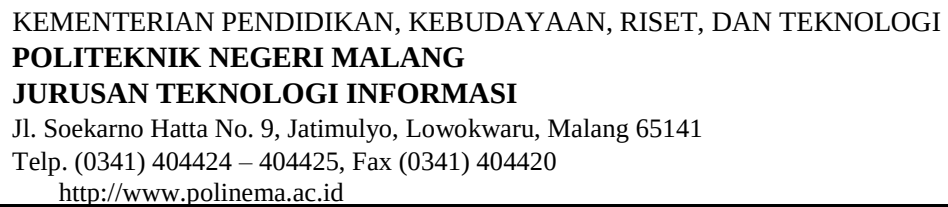
none **form-data** x-www-form-urlencoded raw binary

Key	Value	...	Bulk Edit
☒ username	penggunasatu		
☒ password	12345		
Key	Value		

body Cookies Headers (11) Test Results Status: 200 OK Time: 1501 ms Size: 986 B Save Response

Pretty Raw Preview Visualize JSON

```
1  {
2    "success": true,
3    "user": {
4      "user_id": 17,
5      "level_id": 2,
6      "username": "penggunasatu",
7      "nama": "Pengguna 1",
8      "password": "$2y$12$Eb2SxV1jsykINytYGtrHi0DVAKcK5p6EgnZnmbChkPicIu7S0QJJJu",
9      "created_at": "2024-04-22T15:56:04.000000Z",
10     "updated_at": "2024-04-22T15:56:04.000000Z"
11   },
12   "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vbG9jYXRob3N0L1BXTF9QT1MtbnVpbi9wdWJsaWVYbPL2xvZ21uIiwiaWF0Ij0i"
13 }
```

[illegible]

The screenshot shows a web client interface. At the top, a request is configured with the method **POST** and the URL **http://127.0.0.1:8000/api/login**. The **Body** tab is selected, showing **form-data** with two fields: **username** (value: **AdminNana**) and **password** (value: **12345678**). The response status is **401 Unauthorized** with a duration of **539 ms** and a size of **381 B**. The response body is shown in JSON format:

```
{
  "success": false,
  "message": "Username atau Password Anda salah"
}
```

Hal. 20 / 46



Jawab : API login tidak mendukung metode GET. Rute `api/login` telah dikonfigurasi untuk hanya menerima permintaan POST, sehingga penggunaan GET akan menghasilkan error 405 "Method Not Allowed".

GET `http://127.0.0.1:8000/api/login` Send

Params Auth Headers (9) Body • Scripts Settings Cookies

form-data

☒	username	Text	AdminClaudya
☒	password	Text	12345678

Body `405 Method Not Allowed` 3.97 s • 1022.22 KB

HTML Preview Visualize

```
1 <!DOCTYPE html>
2 <html lang="en" class="auto">
3 <!--
4 Symfony\Component\HttpKernel\Exception\MethodNotAllowedHttpException: The GET method is not supported for route api/login. Supported methods: POST. in file
  C:\laragon\www\PWL2025\Week10\Jobsheet10\vendor\laravel\framework\src\Illuminate\Routing\AbstractRouteCollection.php
  on line 122
5
6 #0 C:\laragon\www\PWL2025\Week10\Jobsheet10\vendor\laravel\fram
```

8. Lakukan commit perubahan file pada Github.

Praktikum 3 – Membuat RESTful API Logout

1. Tambahkan kode berikut pada file `.env`

`JWT_SHOW_BLACKLIST_EXCEPTION=true`

```
61 JWT_SECRET=5nXwIhNTEwy6puOtFAnCK0YZXPERLi13cIG8GdM1rTLmk6azG9F0yPsEy6JTty6ah
62 JWT_SHOW_BLACKLIST_EXCEPTION=true
```

2. Buat Controller baru dengan nama LogoutController. `php artisan make:controller Api/LogoutController`



```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/Logout  
Controller
```

```
INFO Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\  
Api\LogoutController.php] created successfully.
```

3. Buka file tersebut dan ubah kode menjadi seperti berikut.

```
1  <?php
2
3  namespace App\Http\Controllers\Api;
4  use Illuminate\Http\Request;
5  use App\Http\Controllers\Controller;
6  use Tymon\JWTAuth\Facades\JWTAuth;
7  use Tymon\JWTAuth\Exceptions\JWTException;
8  use Tymon\JWTAuth\Exceptions\TokenExpiredException;
9  use Tymon\JWTAuth\Exceptions\TokenInvalidException;
10
11 class LogoutController extends Controller
12 {
13     public function __invoke(Request $request)
14     {
15         //remove token
16         $removeToken = JWTAuth::invalidate(JWTAuth::getToken());
17
18         if($removeToken) {
19             //return response JSON
20             return response()->json([
21                 'success' => true,
22                 'message' => 'Logout Berhasil!',
23             ]);
24         }
25     }
26 }
```

Hasil :



```
app > Http > Controllers > Api > LogoutController.php > PHP > LogoutController
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use Illuminate\Http\Request;
6  use App\Http\Controllers\Controller;
7  use Tymon\JWTAuth\Facades\JWTAuth;
8  use Tymon\JWTAuth\Exceptions\JWTException;
9  use Tymon\JWTAuth\Exceptions\TokenExpiredException;
10 use Tymon\JWTAuth\Exceptions\TokenInvalidException;
11
12 1 reference | 0 implementations
13 class LogoutController extends Controller
14 {
15     0 references | 0 overrides
16     public function __invoke(Request $request): JsonResponse|mixed
17     {
18         //remove token
19         $removeToken = JWTAuth::invalidate(JWTAuth::getToken());
20
21         if($removeToken) {
22             //return response JSON
23             return response()->json([
24                 'success' => true,
25                 'message' => 'Logout Berhasil!',
26             ]);
27         }
28     }
29 }
```

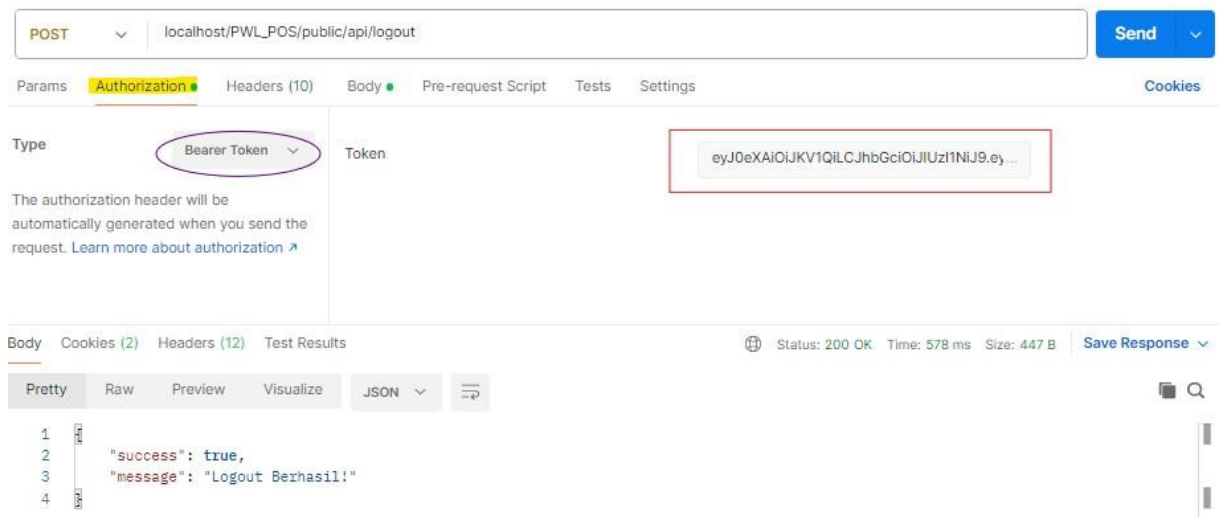
4. Lalu kita tambahkan routes pada api.php

```
Route::post('/logout', App\Http\Controllers\Api\LogoutController::class)->name('logout');
```

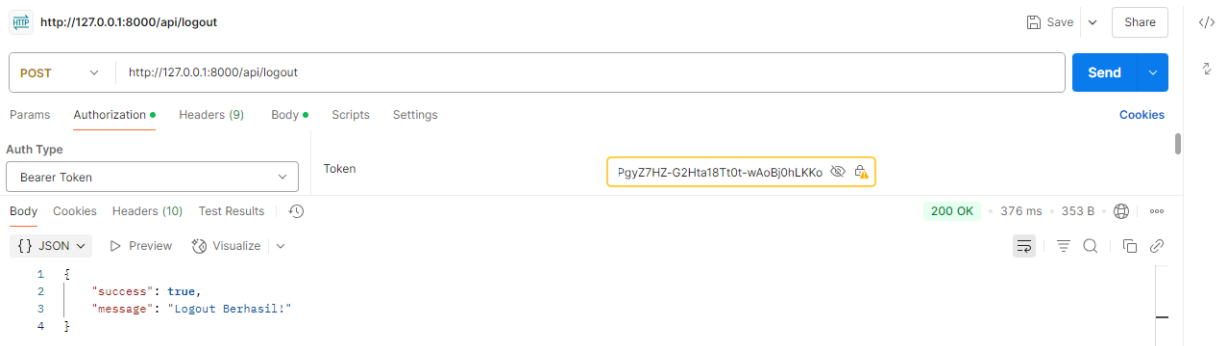
Hasil :

```
Route::post('/logout', App\Http\Controllers\Api\LogoutController::class)->name('logout');
```

5. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman. Buka aplikasi Postman, isi URL localhost/PWL_POS/public/api/logout serta method POST.
6. Isi token pada tab Authorization, pilih Type yaitu Bearer Token. Isikan token yang didapat saat login. Jika sudah klik Send.



Lakukan percobaan yang sama dan berikan screenshot hasil percobaan Anda.



7. Lakukan commit perubahan file pada Github.

Praktikum 4 – Implementasi CRUD dalam RESTful API

Pada praktikum ini kita akan menggunakan tabel m_level untuk dimodifikasi menggunakan RESTful API.

1. Pertama, buat controller untuk mengolah API pada data level.

`php artisan make:controller Api/LevelController`

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/LevelController

INFO Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\LevelController.php] created successfully.
```

2. Setelah berhasil, buka file tersebut dan tuliskan kode seperti berikut yang berisi fungsi CRUDnya.



```
namespace App\Http\Controllers\Api;
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\LevelModel;

class LevelController extends Controller
{
    public function index()
    {
        return LevelModel::all();
    }
}
```

```
public function store(Request $request)
{
    $level = LevelModel::create($request->all());
    return response()->json($level, 201);
}

public function show(LevelModel $level)
{
    return LevelModel::find($level);
}

public function update(Request $request, LevelModel $level)
{
    $level->update($request->all());
    return LevelModel::find($level);
}

public function destroy(LevelModel $user)
{
    $user->delete();

    return response()->json([
        'success' => true,
        'message' => 'Data terhapus',
    ]);
}
}
```

Hasil :



```
app > Http > Controllers > Api > LevelController.php > PHP > LevelController
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use Illuminate\Http\Request;
7  use App\Models\LevelModel;
8
9  6 references | 0 implementations
10 class LevelController extends Controller
```

```
11 public function index(): Collection
12 {
13     return LevelModel::all();
14 }
15
16 1 reference | 0 overrides
17 public function store(Request $request): JsonResponse|mixed
18 {
19     $level = LevelModel::create($request->all());
20     return response()->json($level, 201);
21 }
22
23 1 reference | 0 overrides
24 public function show(LevelModel $level): array|Collection|LevelModel|Model|null
25 {
26     return LevelModel::find($level);
27 }
28
29 1 reference | 0 overrides
30 public function update(Request $request, LevelModel $level): array|Collection|LevelModel|Model|null
31 {
32     $level->update($request->all());
33     return LevelModel::find($level);
34 }
35
36 1 reference | 0 overrides
37 public function destroy(LevelModel $user): JsonResponse|mixed
38 {
39     $user->delete();
40
41     return response()->json([
42         'success' => true,
43         'message' => 'Data terhapus',
44     ]);
45 }
```

3. Kemudian kita lengkapi routes pada api.php.

```
use App\Http\Controllers\Api\LevelController;

Route::get('levels', [LevelController::class, 'index']);
Route::post('levels', [LevelController::class, 'store']);
Route::get('levels/{level}', [LevelController::class, 'show']);
Route::put('levels/{level}', [LevelController::class, 'update']);
Route::delete('levels/{level}', [LevelController::class, 'destroy']);
```

Hasil :



```
29 Route::get('levels', [LevelController::class, 'index']);
30 Route::post('levels', [LevelController::class, 'store']);
31 Route::get('levels/{level}', [LevelController::class, 'show']);
32 Route::put('levels/{level}', [LevelController::class, 'update']);
33 Route::delete('levels/{level}', [LevelController::class, 'destroy']);
```

4. Jika sudah. Lakukan uji coba API mulai dari fungsi untuk menampilkan data. Gunakan URL: `localhost/PWL_POS-main/public/api/levels` dan method GET. **Jelaskan dan berikan screenshot hasil percobaan Anda.**

GET `http://127.0.0.1:8000/api/levels` Send

Params Auth Headers (9) Body Scripts Settings Cookies

Query Params

	Key	Value	Descri...	Bulk Edit
--	-----	-------	-----------	-----------

Body 200 OK • 378 ms • 978 B • [Globe Icon] [More Icon]

[JSON] Preview Visualize [Icons]

```
1  [
2    {
3      "level_id": 1,
4      "level_kode": "ADM",
5      "level_nama": "Administrator",
6      "created_at": null,
7      "updated_at": null
8    },
9    {
10     "level_id": 2,
11     "level_kode": "MNG",
12     "level_nama": "Manager",
13     "created_at": null,
14     "updated_at": null
15   },
16 ]
```



GET Send

Params Auth Headers (9) Body Scripts Settings Cookies

Query Params

Key	Value	Description
level_kode	ADM	
level_nama	Administrator	

Body JSON Preview Visualize

```
1 [
2   {
3     "level_id": 1,
4     "level_kode": "ADM",
5     "level_nama": "Administrator",
6     "created_at": null,
7     "updated_at": null
8   },
9 ]
```

5. Kemudian, lakukan percobaan penambahan data dengan URL : `localhost/PWL_POSmain/public/api/levels` dan method POST seperti di bawah ini.

POST Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description
level_kode	Text SPV	
level_nama	Text Supervisor	

Body Cookies Headers (11) Test Results Status: 201 Created Time: 276 ms Size: 531 B Save as example

Pretty Raw Preview Visualize JSON

```
1 {
2   "level_kode": "SPV",
3   "level_nama": "Supervisor",
4   "updated_at": "2024-04-22T21:40:32.000000Z",
5   "created_at": "2024-04-22T21:40:32.000000Z",
6   "level_id": 4
7 }
```

Jelaskan dan berikan screenshoot hasil percobaan Anda.



POST ▼ <http://127.0.0.1:8000/api/levels> Send ▼

Params Auth ● Headers (9) Body ● Scripts Settings Cookies

form-data ▼

	Key		Value	Descrip...	...	Bulk Edit
☒	level_kode	Text ▼	SPV			
☒	level_nama	Text ▼	Supervisor			
	Key	Text ▼	Value	Description		

Body ▼ ↺ 201 Created • 594 ms • 459 B • 🌐 | ...

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ ☰ ☷ 🔍 📄 🔗

```
1 {
2   "level_kode": "SPV",
3   "level_nama": "Supervisor",
4   "updated_at": "2025-04-30T14:44:29.000000Z",
5   "created_at": "2025-04-30T14:44:29.000000Z",
6   "level_id": 15
7 }
```

Data Tambahkan:

7	SPV	Supervisor	<button>Detail</button> <button>Edit</button> <button>Hapus</button>
---	-----	------------	--

6. Berikutnya lakukan percobaan menampilkan detail data. **Jelaskan dan berikan screenshot hasil percobaan Anda.**



GET ▼ <http://127.0.0.1:8000/api/levels> Send ▼

Params Auth ● Headers (9) Body ● Scripts Settings Cookies

form-data ▼

	Key		Value	Descrip...	...	Bulk Edit
☒	level_kode	Text ▼	SPV			
☒	level_nama	Text ▼	Supervisor			
	Key	Text ▼	Value	Description		

Body ▼ 🔄 200 OK • 414 ms • 1.1 KB • 🌐 ...

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
42     "updated_at": null
43   },
44   {
45     "level_id": 15,
46     "level_kode": "SPV",
47     "level_nama": "Supervisor",
48     "created_at": "2025-04-30T14:44:29.000000Z",
49     "updated_at": "2025-04-30T14:44:29.000000Z"
50   }
51 ]
```

7. Jika sudah, kita coba untuk melakukan edit data menggunakan `localhost/PWL_POSmain/public/api/levels/{id}` dan method PUT. Isikan data yang ingin diubah pada tab Param.



PUT localhost/PWL_POS-main/public/api/levels/4?level_kode=SPR Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

☒	Key	Value	Description	*** Bulk Edit
☒	level_kode	SPR		
	Key	Value	Description	

body Cookies Headers (11) Test Results Status: 200 OK Time: 266 ms Size: 528 B Save as example

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "level_id": 4,
4     "level_kode": "SPR",
5     "level_nama": "Supervisor",
6     "created_at": "2024-04-22T21:40:32.000000Z",
7     "updated_at": "2024-04-22T21:48:19.000000Z"
8   }
9 ]
```

Jelaskan dan berikan screenshoot hasil percobaan Anda.

PUT http://127.0.0.1:8000/api/levels/15?level_kode=SPR&le... Send

Params Auth Headers (9) Body Scripts Settings Cookies

Query Params

☒	Key	Value	Descrip...	*** Bulk Edit
☒	level_kode	SPR		
☒	level_nama	Supervisor		
	Key	Value	Description	

Body 200 OK 1.15 s 454 B

{ JSON Preview Visualize

```
1 {
2   "level_id": 15,
3   "level_kode": "SPR",
4   "level_nama": "Supervisor",
5   "created_at": "2025-04-30T14:44:29.000000Z",
6   "updated_at": "2025-04-30T15:11:28.000000Z"
7 }
```

7	SPR	Supervisor	Detail Edit Hapus

8. Terakhir lakukan percobaan hapus data. Jelaskan dan berikan screenshoot hasil percobaan Anda.



DELETE

http://127.0.0.1:8000/api/levels/15

Send

Params Auth Headers (9) Body Scripts Settings Cookies

Query Params

	Key	Value	Descrip...	Bulk Edit
	Key	Value	Description	

Body 200 OK • 373 ms • 350 B

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Data terhapus"
4 }
```

No	Kode Level	Nama Level	Aksi
1	ADM	Administrator	Detail Edit Hapus
2	MNG	Manager	Detail Edit Hapus
3	STF	Staff/Kasir	Detail Edit Hapus
4	CUS2	Customer	Detail Edit Hapus
5	PGR	Programer	Detail Edit Hapus
6	HRD	Human Resource Development	Detail Edit Hapus

Showing 1 to 6 of 6 entries

Previous 1 Next

9. Lakukan commit perubahan file pada Github.



TUGAS

Implementasikan CRUD API pada tabel lainnya yaitu tabel m_user, m_kategori, dan m_barang

1. API pada m_user

- UserController

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/UserController
```

```
INFO Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\UserController.php] created successfully.
```

```
app > Http > Controllers > Api > UserController.php > PHP > UserController
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use Illuminate\Http\Request;
7  use App\Models\UserModel;
8  use Illuminate\Http\JsonResponse;
9
10 1 reference | 0 implementations
11 class UserController extends Controller
```



```
0 references | 0 overrides
12 public function index(): Collection
13 {
14     return UserModel::all();
15 }
16
0 references | 0 overrides
17 public function store(Request $request): JsonResponse|mixed
18 {
19     $user = UserModel::create($request->all());
20     return response()->json(["data" => $user, "status" => 201]);
21 }
22
0 references | 0 overrides
23 public function show(UserModel $user): UserModel
24 {
25     return $user;
26 }
27
0 references | 0 overrides
28 public function update(Request $request, UserModel $user): UserModel
29 {
30     $user->update($request->all());
31     return $user;
32 }
33
0 references | 0 overrides
34 public function destroy(UserModel $user): JsonResponse|mixed
35 {
36     $user->delete();
37     return response()->json([
38         "success" => true,
39         "message" => "Data terhapus"
40     ]);
41 }
42 }
```

- Route

```
25 // API User
26 Route::get('/users', [UserController::class, 'index']);
27 Route::post('/users', [UserController::class, 'store']);
28 Route::get('/users/{user}', [UserController::class, 'show']);
29 Route::put('/users/{user}', [UserController::class, 'update']);
30 Route::delete('/users/{user}', [UserController::class, 'destroy']);
```

- CRUD

- create



POST ▼ <http://127.0.0.1:8000/api/users> Send ▼

Params Auth ● Headers (9) Body ● Scripts Settings Cookies

form-data ▼

☒	level_id	Text ▼	1	
☒	username	Text ▼	AdmSatu	
☒	nama	Text ▼	Miya	
☒	password	Text ▼	12345	
	Key	Text ▼	Value	Description

Body ▼ 🔄 200 OK 958 ms 480 B 🌐 ⋮

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
2   "data": {
3     "level_id": "1",
4     "username": "AdmSatu",
5     "nama": "Miya",
6     "updated_at": "2025-04-30T15:52:22.000000Z",
7     "created_at": "2025-04-30T15:52:22.000000Z",
8     "user_id": 35
9   },
```

19	AdmSatu	Miya	Administrator	Detail Edit Hapus
----	---------	------	---------------	--

○ Read

GET ▼ <http://127.0.0.1:8000/api/users/35> Send ▼

Params Auth ● Headers (9) Body ● Scripts Settings Cookies

Query Params

	Key	Value	Descrip...	***	Bulk Edit
	Key	Value	Description		

Body ▼ 🔄 200 OK 620 ms 484 B 🌐 ⋮

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
1  {
2    "user_id": 35,
3    "level_id": 1,
4    "username": "AdmSatu",
5    "nama": "Miya",
6    "user_profile_picture": null,
7    "created_at": "2025-04-30T15:52:22.000000Z",
8    "updated_at": "2025-04-30T15:52:22.000000Z"
9  }
```



○ Update

PUT ▼ <http://127.0.0.1:8000/api/users/35?nama=Miya Lolita> Send ▼

Params ● Auth ● Headers (9) ● Body ● Scripts Settings Cookies

Query Params

☒	Key	Value	Descrip...	*** Bulk Edit
☒	nama	Miya Lolita		
	Key	Value	Description	

Body ▼ 🔄 200 OK 950 ms 491 B 🌐 ⋮

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
1 {
2   "user_id": 35,
3   "level_id": 1,
4   "username": "AdmSatu",
5   "nama": "Miya Lolita",
6   "user_profile_picture": null,
7   "created_at": "2025-04-30T15:52:22.000000Z",
8   "updated_at": "2025-04-30T15:57:24.000000Z"
9 }
```

19	AdmSatu	Miya Lolita	Administrator	Detail	Edit	Hapus

○ Delete

DELETE ▼ <http://127.0.0.1:8000/api/users/35> Send ▼

Params ● Auth ● Headers (9) ● Body ● Scripts Settings Cookies

Query Params

	Key	Value	Descrip...	*** Bulk Edit
	Key	Value	Description	

Body ▼ 🔄 200 OK 500 ms 350 B 🌐 ⋮

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
1 {
2   "success": true,
3   "message": "Data terhapus"
4 }
```




2. API pada m_kategori

- KategoriController

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/KategoriController
```

INFO Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\KategoriController.php] created successfully.

```
app > Http > Controllers > Api > KategoriController.php > PHP > KategoriController
```

```
1 <?php
2
3 namespace App\Http\Controllers\Api;
4
5 use App\Http\Controllers\Controller;
6 use Illuminate\Http\Request;
7 use App\Models\KategoriModel;
8 use Illuminate\Database\Eloquent\Collection;
9 use Illuminate\Http\JsonResponse;
10
11 class KategoriController extends Controller
12
```



```
13     public function index(): Collection
14     {
15         return KategoriModel::all();
16     }
17
18     0 references | 0 overrides
19     public function store(Request $request): JsonResponse|mixed
20     {
21         $kategori = KategoriModel::create($request->all());
22         return response()->json($kategori, 201);
23     }
24
25     0 references | 0 overrides
26     public function show(KategoriModel $kategori): KategoriModel
27     {
28         return $kategori;
29     }
30
31     0 references | 0 overrides
32     public function update(Request $request, KategoriModel $kategori): KategoriModel
33     {
34         $kategori->update($request->all());
35         return $kategori;
36     }
37
38     0 references | 0 overrides
39     public function destroy(KategoriModel $kategori): JsonResponse|mixed
40     {
41         $kategori->delete();
42
43         return response()->json([
44             'success' => true,
45             'message' => 'Data kategori terhapus',
46         ]);
47     }
```

- Routes

```
use App\Http\Controllers\Api\KategoriController;
```

```
// API Kategori
Route::get('/kategoris', [KategoriController::class, 'index']);
Route::post('/kategoris', [KategoriController::class, 'store']);
Route::get('/kategoris/{kategori}', [KategoriController::class, 'show']);
Route::put('/kategoris/{kategori}', [KategoriController::class, 'update']);
Route::delete('/kategoris/{kategori}', [KategoriController::class, 'destroy']);
```

- CRUD

- Create



POST <http://127.0.0.1:8000/api/kategoris>

Params Auth Headers (9) **Body** Scripts Settings Cookies

form-data

	Key		Value	Descrip...	Bulk Edit
☒	kategori_kode	Text	BTK		
☒	kategori_nama	Text	Batik		
	Key	Text	Value	Description	

Body 201 Created • 480 ms • 463 B •

```
1 {
2   "kategori_kode": "BTK",
3   "kategori_nama": "Batik",
4   "updated_at": "2025-04-30T16:11:08.000000Z",
5   "created_at": "2025-04-30T16:11:08.000000Z",
6   "kategori_id": 29
7 }
```

	Key		Value	Descrip...	Bulk Edit
☒	kategori_kode	Text	BTK		
☒	kategori_nama	Text	Batik		
	Key	Text	Value	Description	

10 BTK Batik

○ Read

GET <http://127.0.0.1:8000/api/kategoris/29>

Params Auth Headers (9) **Body** Scripts Settings Cookies

form-data

	Key		Value	Descrip...	Bulk Edit
☒	kategori_kode	Text	BTK		
☒	kategori_nama	Text	Batik		
	Key	Text	Value	Description	

Body 200 OK • 371 ms • 458 B •

```
1 {
2   "kategori_id": 29,
3   "kategori_kode": "BTK",
4   "kategori_nama": "Batik",
5   "created_at": "2025-04-30T16:11:08.000000Z",
6   "updated_at": "2025-04-30T16:11:08.000000Z"
7 }
```



○ Update

PUT ▼ http://127.0.0.1:8000/api/kategoris/29?kategori_nama ... Send ▼

Params ● Auth ● Headers (9) ● Body ● Scripts Settings ● Cookies

Query Params

☒	Key	Value	Descrip...	...	Bulk Edit
☒	kategori_nama	Batik Tulis			
	Key	Value	Description		

Body ▼ 🔄 200 OK • 427 ms • 464 B • 🌐 ...

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
1 {  
2   "kategori_id": 29,  
3   "kategori_kode": "BTK",  
4   "kategori_nama": "Batik Tulis",  
5   "created_at": "2025-04-30T16:11:08.000000Z",  
6   "updated_at": "2025-04-30T16:13:34.000000Z"  
7 }
```

ID	Kode	Nama	Aksi
10	BTK	Batik Tulis	Detail Edit Hapus

○ Delete

DELETE ▼ <http://127.0.0.1:8000/api/kategoris/29> Send ▼

Params ● Auth ● Headers (9) ● Body ● Scripts Settings ● Cookies

Query Params

	Key	Value	Descrip...	...	Bulk Edit
	Key	Value	Description		

Body ▼ 🔄 200 OK • 612 ms • 359 B • 🌐 ...

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
1 {  
2   "success": true,  
3   "message": "Data kategori terhapus"  
4 }
```



3. API pada m_barang

- BarangController

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/BarangController

INFO Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\BarangController.php] created successfully.
```

```
app > Http > Controllers > Api > BarangController.php > PHP > BarangController

1 <?php
2
3 namespace App\Http\Controllers\Api;
4
5 use App\Http\Controllers\Controller;
6 use Illuminate\Http\Request;
7 use App\Models\BarangModel;
8 use Illuminate\Database\Eloquent\Collection;
9 use Illuminate\Http\JsonResponse;
10
11 1 reference | 0 implementations
12 class BarangController extends Controller
13 {
```



```
13 public function index(): Collection
14 {
15     return BarangModel::all();
16 }
17
18 0 references | 0 overrides
19 public function store(Request $request): JsonResponse|mixed
20 {
21     $barang = BarangModel::create($request->all());
22     return response()->json([
23         'data' => $barang,
24         'status' => 201
25     ]);
26 }
27
28 0 references | 0 overrides
29 public function show(BarangModel $barang): BarangModel
30 {
31     return $barang;
32 }
33
34 0 references | 0 overrides
35 public function update(Request $request, BarangModel $barang): BarangModel
36 {
37     $barang->update($request->all());
38     return $barang;
39 }
40
41 0 references | 0 overrides
42 public function destroy(BarangModel $barang): JsonResponse|mixed
43 {
44     $barang->delete();
45     return response()->json([
46         'message' => 'Data terhapus.'
47     ], 200);
48 }
```

- Routes

```
use App\Http\Controllers\Api\BarangController;

// API Barang
Route::get('/barangs', [BarangController::class, 'index']);
Route::post('/barangs', [BarangController::class, 'store']);
Route::get('/barangs/{barang}', [BarangController::class, 'show']);
Route::put('/barangs/{barang}', [BarangController::class, 'update']);
Route::delete('/barangs/{barang}', [BarangController::class, 'destroy']);
```

- CRUD

- Create



POST ▼ <http://127.0.0.1:8000/api/barangs> Send ▼

Params Auth ● Headers (9) Body ● Scripts Settings Cookies

form-data ▼

☒	barang_nama	Text ▼	Parfum	
☒	harga_beli	Text ▼	80000	
☒	harga_jual	Text ▼	90000	
☒	kategori_id	Text ▼	4	
	Key	Text ▼	Value	Description

Body ▼ 🕒 200 OK • 576 ms • 537 B • 🌐 ⋮

{} JSON ▼ ▶ Preview 🔍 Visualize ▼ 🔍 🔍 🔍 🔍 🔍

```
1 {
2   "data": {
3     "barang_kode": "BRG15",
4     "barang_nama": "Parfum",
5     "harga_beli": "80000",
6     "harga_jual": "90000",
7     "kategori_id": "4",
8     "updated_at": "2025-04-30T16:29:07.000000Z",
9     "created_at": "2025-04-30T16:29:07.000000Z",
10    "barang_id": 21
11  }
```

17	4	BRG15	Parfum	80.000	90.000	Pewangi	Detail Edit Hapus
----	---	-------	--------	--------	--------	---------	--

○ Read



GET Send

Params Auth Headers (9) **Body** Scripts Settings Cookies

form-data

☒	barang_nama	Text	Parfum	
☒	harga_beli	Text	80000	
☒	harga_jual	Text	90000	
☒	kategori_id	Text	4	
	Key	Text	Value	Description

Body 200 OK • 686 ms • 509 B

{ } JSON Preview Visualize

```
1 {
2   "barang_id": 21,
3   "kategori_id": 4,
4   "barang_kode": "BRG15",
5   "barang_nama": "Parfum",
6   "harga_beli": 80000,
7   "harga_jual": 90000,
8   "created_at": "2025-04-30T16:29:07.000000Z",
9   "updated_at": "2025-04-30T16:29:07.000000Z"
10 }
```

○ Update



PUT http://127.0.0.1:8000/api/barangs/21?barang_nama=P...

Params ☒ Auth ☒ Headers (9) Body ☒ Scripts Settings

[Cookies](#)

Query Params

☒	Key	Value	Descrip...	*** Bulk Edit
☒	barang_nama	Parfum Pakaian		
	Key	Value	Description	

Body ☒ 200 OK = 1.07 s = 517 B

```
{
  "barang_id": 21,
  "kategori_id": 4,
  "barang_kode": "BRG15",
  "barang_nama": "Parfum Pakaian",
  "harga_beli": 80000,
  "harga_jual": 90000,
  "created_at": "2025-04-30T16:29:07.000000Z",
  "updated_at": "2025-04-30T16:32:07.000000Z"
}
```

17	4	BRG15	Parfum Pakaian	80.000	90.000	Pewangi	Detail Edit Hapus
----	---	-------	----------------	--------	--------	---------	---

○ Delete

DELETE <http://127.0.0.1:8000/api/barangs/21>

Params ☒ Auth ☒ Headers (9) Body ☒ Scripts Settings

[Cookies](#)

Query Params

	Key	Value	Descrip...	*** Bulk Edit
	Key	Value	Description	

No	ID Kategori	Kode Barang	Nama Barang	Harga Beli	Harga Jual	Kategori	Aksi
11	2	BRG011	lip cream	15.000	18.000	MakeUp	Detail Edit Hapus
12	1	SBK-003	Telur Omega(10 butir)	22.000	25.000	Makanan	Detail Edit Hapus
13	2	SNK-003	Sari Roti	11.500	12.500	MakeUp	Detail Edit Hapus
14	3	MND-003	Shampo Pantene	17.500	18.500	Aksesoris	Detail Edit Hapus
15	4	BAY-003	Baju Bayi 2th	89.000	92.500	Pewangi	Detail Edit Hapus
16	5	MNM-003	Cleo 600ml	3.750	4.300	Alat Mandi	Detail Edit Hapus

Showing 11 to 16 of 16 entries

[Previous](#) [1](#) [2](#) [Next](#)



KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<http://www.polinema.ac.id>

**** Sekian, dan selamat belajar ****